

Video Summary

This Telusko lesson explains modules as containers for related functions. It shows how the built-in math module helps with advanced calculations through `sqrt()`, `ceil()`, `pow()`, import styles, aliases, and `help()`.

Source: user-provided [Telusko/Navin Reddy Day 17 video summary](#)

1

Why Modules Matter

A module groups useful code under one name. Instead of keeping every function in one file, Python lets us import tools when we need them.

box

Organized

Related functions stay grouped together under a clear module name.

math

Ready-made

Python already includes useful modules for common work.

import

Professional

Use trusted tools instead of rewriting complex logic from scratch.

```
>>> import math
>>> math.sqrt(25)
5.0
>>> math.ceil(4.2)
5
```

quick module example

2

Modules Are Code Containers

Think of a module as a box that stores related Python functions, constants, and other tools. When a program needs one of those tools, it imports the module and calls the function from that box.

file

One file

A module can be a Python file that holds reusable code.

tools

Many tools

A module can contain many functions that solve related tasks.

.

Dot access

Use `module.function()` to call a function from that module.

Without modules	With modules	Why it helps
Write every formula yourself.	Import trusted code.	Less repeated work.
Keep many tasks in one place.	Group related tools.	Cleaner program structure.
Remember all details manually.	Call named functions.	Easier to read and maintain.

```
# Python keeps useful tools in modules.
import math
answer = math.sqrt(64)
print(answer)
Output:
8.0
```

module idea

3

Use import math

A full import loads the module name into your program. After that, call functions using `math.function_name()`. This style is clear because the code shows exactly where the function came from.

import

Import first

Write `import math` near the top before using math tools.

math.

Use prefix

Call the function with the module prefix, such as `math.sqrt()`.

sqrt

Square root

`sqrt()` finds the number that multiplies by itself to make the input.

```
import math

number = 36
root = math.sqrt(number)
print(root)

Output:
6.0
```

Expression	Meaning	Result
<code>math.sqrt(25)</code>	Square root of 25.	5.0
<code>math.sqrt(2)</code>	Square root of 2.	1.414...
<code>math.sqrt(0)</code>	Square root of zero.	0.0

Beginner habit

Use the full import style when learning. The `math.` prefix makes the origin of `sqrt()`, `ceil()`, and `pow()` easy to understand.

4

Advanced Calculations

The math module gives ready-made functions for calculations that are more advanced than simple +, -, *, and /. These functions save time and reduce mistakes.

ceil**Round up**

`math.ceil(4.2)` returns the next whole number: 5.

floor**Round down**

`math.floor(4.9)` returns the lower whole number: 4.

pow**Power**

`math.pow(3, 2)` calculates 3 raised to power 2.

Function	Purpose	Example result
<code>math.ceil(4.2)</code>	Round up to the nearest integer.	5
<code>math.floor(4.9)</code>	Round down to the nearest integer.	4
<code>math.pow(3, 2)</code>	Raise 3 to the power of 2.	9.0
<code>math.sqrt(49)</code>	Find a square root.	7.0

```
math function practice

>>> import math
>>> math.ceil(6.3)
7
>>> math.floor(6.9)
6
>>> math.pow(2, 5)
32.0
```

5

Choose an Import Style

Python gives more than one way to import module tools. Choose the style that keeps the program clear, short, and easy to understand.

Style	Code	How you call it
Full import	<code>import math</code>	<code>math.sqrt(25)</code>
Selective import	<code>from math import sqrt</code>	<code>sqrt(25)</code>
Alias import	<code>import math as m</code>	<code>m.sqrt(25)</code>

```
import math
print(math.sqrt(25))

from math import sqrt
print(sqrt(25))

import math as m
print(m.ceil(4.2))

Output:
5.0 5.0 5
```

math**Full**

Best for clarity because every call shows the module name.

sqrt**Selective**

Useful when you only need a small number of functions.

m**Alias**

Helpful when a module name is long or used very often.

Style rule

Avoid mixing too many import styles in one small program. Consistent imports make code easier to read.

6

Use help() to Explore Modules

Python can show documentation from the terminal. Use `help(math)` to see the module overview, or `help(math.ceil)` to inspect one function.

```
terminal help

>>> import math
>>> help(math)
# Shows module documentation.
>>> help(math.ceil)
# Shows one function in detail.
>>> dir(math)
# Lists names available inside math.
```

Practice	Question to ask	Good answer
Import	Did you import the module first?	Use <code>import math</code> before <code>math.sqrt()</code> .
Prefix	Does the call show where it comes from?	Use <code>math.function_name()</code> for full import.
Need	Do you need one tool or many?	Use selective import only when it stays clear.
Docs	Are you unsure what a function does?	Try <code>help(math.ceil)</code> in the terminal.

Day 17/55 takeaway

Modules group useful tools. Use `import math` for ready-made calculations, practice `sqrt()`, `ceil()`, and `pow()`, and open `help()` when you need documentation.